PROJECT-BASED LEARNING

PHASE-III FINAL EVALUATION

MiniGit: A Lightweight Version Control System Using C++ and Data Structures

Team ID: DSCPP-III-2026-T296

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T296
Semester: 3rd
Section: ML-1,2,3,4
Domain: CSE (AI&ML)

Team Members

1. Gangwar, Abhinesh – 2510370192 (Team Lead)
2. Jain, Sparsh – 2510370159
3. Bajaj, Krish – 2510370469
4. Joshi, Snehil – 2510370146

Mentor

Mr. Himanshu Mainwal

Interactions: _____

Project Abstract & Problem Statement

Project Abstract

A local, command-line version-control system in C++ where each operation rests on a classic data structure (Dual Trie, ordered set, deques, doubly linked list, commit DAG). Phase III added restore (`checkout`, `rollback`), branches and the commit graph; all of it is merged, ships as one executable and passes 13 test suites.

Problem Statement

Full version-control systems are large and hard to read. MiniGit is a small, readable tool to create a repository, track files, save versions, view history and restore earlier work.

Target Users / Application

- Students and learners studying version control and data structures
- Local projects of text files on Windows, from any terminal
- Benefit: a working tool whose every command maps to code a student can read

Objectives & Final Deliverables

Finalized Objectives

1. Create a repository and stage files: `init`, `add`, `remove`
2. Apply `.minigitignore` rules with a Dual Trie
3. Save commits with SHA-1 ids, metadata, parents and snapshots
4. View and manage history: `log`, `undo`, `redo`
5. Restore work: `checkout <commit-id>`, `rollback`
6. Branches and a commit graph: `branch`, `log --all --graph`

Final Deliverables

- `minigit.exe`: one standalone executable, first-run `install`
- Organized C++ source (`module/include`, `module/src`)
- 13 automated test suites
- Sample repository with a step-by-step demo walkthrough
- README, `DOCUMENTATION.md`, documentation site
- Phase III report and live demonstration

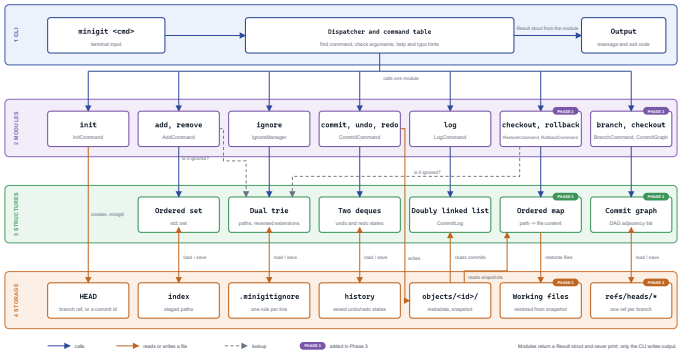
Phase-I & Phase-II Feedback Incorporated

Phase	Feedback / Suggestion	Action Taken
Phase-I	Architecture diagram needed correction	Architecture redrawn as five layers
Phase-I	Problem-solving approach needed improvement	Stack replaced by deque; singly linked list made doubly linked
Phase-I	Team self-review: justify data-structure choices	Dual-Trie ignore engine added; hash table replaced by `std::set`
Phase-II	Need to finish the remaining features	Finished `checkout` and `rollback` (6 of 6 core commands); added branches and the commit graph
Phase-II	Prepare a sample repository for demonstration	Added `sample_repo/` with a step-by-step demo walkthrough

Final Improvement

From 4 of 6 core commands in Phase II to 6 of 6, with one merged branch design, one shared commit reader, the commit graph and a demo repository.

Final System Architecture



Flow: terminal command, then the dispatcher, then a command module. Modules return result objects and only the CLI prints; `RepoUtils` is the shared reader for commits and HEAD; every write to `.minigit` is atomic.

Final Technology Stack

Software / Technologies

- Programming: C++14 (MinGW GCC 6.3)
- Framework: none; plain C++ command-line program
- Database: none; flat files in `.minigit`
- Libraries: C++ standard library, bundled `ghc::filesystem`
- OS/Platform: Windows terminal

Development Infrastructure

- IDE: VS Code
- Version Control: Git/GitHub
- Build: `mingw32-make`
- Testing: 13 C++ test executables
- Deployment: one static `minigit.exe`; `install` adds it to PATH

Unchanged from Phase II. SHA-1 is implemented in-house; nothing to install beyond the executable.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C++	Trie (forward + reversed-extension, O(L) per check); ordered set; two deques (limit 20); doubly linked list	Ignore engine, staging, undo/redo, `log`
Data Structures in C++	Graph as adjacency list (DAG); three-colour DFS cycle check; topological order with a priority queue; BFS reachability; hash map; ordered map	Branches, `log --all --graph`, checkout and rollback
OOPs with C++	Classes with private state (`Trie`, `IgnoreManager`, `StagingArea`, `CommitLog`, `CommitGraph`, `CLI`); result structs (`AddResult`, `CommitResult`, `BranchResult`); STL containers and algorithms	Every command module and the CLI

Each concept above is in the code and can be shown in the demo or the source.

Module-Wise Final Implementation

Module	Implementation	Responsible	Status
Setup, CLI, integration	`init`, dispatcher, help and typo hints, `install`; Phase III merge into one checkout command and one commit reader	Abhinesh Gangwar	Completed
Ignore engine and staging	Dual Trie, `IgnoreManager`, `StagingArea`, `add`/`remove`/`ignore`; updated in Phase III	Sparsh Jain	Completed
Commits, rollback, branches	SHA-1 commits, undo/redo; `rollback`, `branch`, `checkout <branch>`, commit DAG and `log --all --graph`; atomic writes	Krish Bajaj	Completed
Log and checkout	`CommitLog`, `log`/`--reverse`; `checkout <commit-id>` with detached HEAD and all-or-nothing restore	Snehil Joshi	Completed

Final Project Completion: 100% of core scope (6 of 6 core commands)
Not built, outside the proposal: merge, remote repositories, diffs

Working System / Prototype Demonstration

Major Working Features

- Stage files; ignored files never staged
- Commit, `undo`, `redo`, `log`
- Branch, switch, delete
- `log --all --graph` over all branches
- `checkout <commit-id>` and `rollback`, refusing if unsaved work would be lost

```
> minigit branch feature
Created branch 'feature' at a29ed98
> minigit checkout feature
Switched to branch 'feature'
> minigit commit Document adding a note
[feature 0ebd2b3] Document adding a note
> minigit checkout main
> minigit log --all --graph
* commit 0ebd2b3 (feature)
|     Document adding a note
* commit a29ed98 (HEAD -> main)
|     Plan note search
* commit 6707c6d
> minigit checkout 6707c6d
HEAD is now at 6707c6d Initial notes app
6 file(s) restored, 0 removed
```

Demo walkthrough output (add and author/date lines trimmed); the live demo runs the same steps.

Data / Database / Backend Implementation

Data / Storage (`.minigit`)

- No database: plain files, fully local
- `HEAD`: branch ref, or a commit id when detached
- `index`: staged paths; `history`: commit order
- `refs/heads/<name>`: one file per branch
- `objects/<id>/`: metadata (message, date, parent, deleted files) and a snapshot of changed files

Backend / Core Logic

- Commit ids: 40-character SHA-1 (RFC 3174)
- Atomic writes: temp file, then rename
- Checkout: all files swapped in with backups; any failure undoes all
- Safety: refuses if unsaved work would be lost; blocks `../` in branch names and stored paths
- Errors: result objects carry the message; non-zero exit code

Detached HEAD: `commit`, `undo`, `redo` and `rollback` refuse and explain how to continue, because they would move a branch.

Testing & Validation

Test Type	Expected Result	Observed Result	Status
Unit (10 suites)	Each module behaves as specified, including failure cases	All suites pass	Pass
Integration	Whole workflows across commands give the right files and HEAD	361 checks pass	Pass
Regression	Old bugs stay fixed; path traversal and damaged repositories rejected	326 checks pass	Pass
System (manual)	Built `minigit.exe`: init to graph, restore by id and branch, refusals while detached	Matches the walkthrough output	Pass

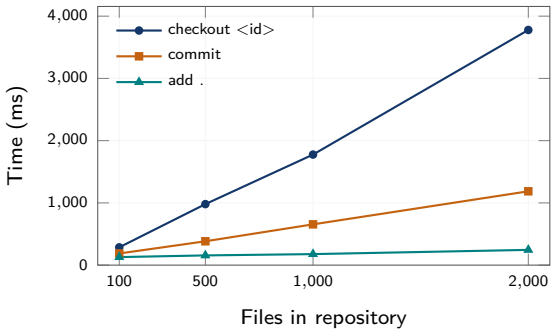
Validation Summary

13 of 13 suites pass on the merged `main` build; every objective is tested, including unsaved work, ignored and deleted files, and damaged commits.

Results & Performance Analysis

Key Results

- 6/6 core commands; 13/13 test suites pass
- One 4.05 MB executable, no external dependency
- 2000 files: `add .` 0.25 s, `commit` 1.2 s, `checkout` 3.8 s
- Time grows about linearly with file count: 1000 to 2000 files roughly doubles commit and checkout



Lab measurement, not field data: one Windows 11 laptop, median of 3 runs, two-line text files, times include process start. Checkout restores every file of the commit, so it is the slowest step.

GitHub Repository & Development Evidence

Repository Details

- Repository: github.com/strange-exe/MiniGit
- Branch: `main` (integrated); one branch per member, plus `ui`
- README.md: updated for Phase III
- Contributors: all 4 members
- Documentation: `DOCUMENTATION.md`, docs site, `Reports/`

Development Evidence

- 30 commits on `main`
- Descriptive messages, mostly `feat:`, `fix:`, `docs:`
- Member work merged from personal branches and pull requests
- Integrated work credited with co-author trailers
- Merged into `main` after the full test suite passes

Requirement: All members must be contributors and individual development activity should be visible through the GitHub history.

Individual Contribution

Member	Role	Final Technical Contribution	Commits
Abhinesh Gangwar	Team Lead	Phase III integration, CLI and dispatcher, `install`, sample repo and walkthrough, docs site	25
Sparsh Jain	Developer	Ignore rules and the modules behind them: Dual Trie, `IgnoreManager`, `StagingArea`, `add/remove/ignore`	5
Krish Bajaj	Developer	`rollback`, `branch`, `checkout <branch>`, commit DAG and `log --all --graph`, atomic writes, tests	6
Snehil Joshi	Developer	`CommitLog` and `log`; `checkout <commit-id>` with detached HEAD and safe restore; checkout tests	7

Challenges, Limitations & Lessons Learned

Challenges & Solutions

- Two members built branch/checkout separately – kept one design, merged the commit-id checkout into it
- Checkout read commits its own way and would revive deleted files – one shared commit reader, plus a test
- Branch code treated a detached HEAD as damage – HEAD may hold a commit id; branch-moving commands refuse
- MinGW returned a different error for missing files – check the file type instead

Current Limitations

- No merge, no diffs, no remote repositories
- Built and tested on Windows only

Technical Learning

- Picking a data structure by its operation cost (Trie O(L), set, DAG)
- Graph algorithms in real use: DFS, BFS, topological sort
- Safe file updates: temp files, backups, all-or-nothing restore

Team Learning

- One owner per module, agreed before coding
- Shared file formats and a full test run before every merge

Final Outcomes, Applications & Future Scope

Final Outcomes

- Working CLI from `init` to `checkout` and `rollback`
- Branches and commit graph
- 13 passing test suites

Applications

- Teaching version control internals
- Worked examples of data structures
- Versioning small local text projects

Future Scope

- Remote repositories (push/pull)
- Line-level diffs
- Rebase

Tangible Project Outcome

A working prototype: one standalone `minigit.exe`, its source code and tests, a sample repository with a demo walkthrough, and a documentation site.

References & Project Resources

References

1. E. Fredkin, "Trie Memory," *Commun. ACM*, vol. 3, no. 9, pp. 490–499, 1960.
2. D. Eastlake 3rd and P. Jones, RFC 3174: US Secure Hash Algorithm 1 (SHA1), 2001.
3. S. Chacon and B. Straub, *Pro Git*, 2nd ed., Apress.
4. P. Morin, *Open Data Structures (in C++)*, opendatastructures.org.
5. cppreference.com, C++ Standard Library reference.
6. Git Documentation, git-scm.com/doc.

Project Resources

- GitHub: github.com/strange-exe/MiniGit
- Demo: live, from `sample_repo/`
- Dataset: NA
- Documentation: abhinesh.codes/MiniGit

Thank You

Questions & Discussion